ITF MANUAL TESTING COURSE

Cinema Database

Designing a Database and Writing SQL Queries

Project overview

Item	Details
Student	Valentina Hatos
Role	QA Tester
Application	MySQL database “cinema”
Module tested	Tables, relationships and SQL queries
Tools	MySQL Workbench
Date	April 2022

Contents

1. Introduction....3

2. Database Model....3

3. Creating the Tables....3

3.1 The movie table....3

3.2 The lead_actor table....3

3.3 The movie_genres table....4

3.4 The screening table....4

4. Changing the Structure....4

5. Queries....4

6. Conclusion....5

1. Introduction

This project is an SQL exercise from the ITF Manual Testing course. I created a database for a cinema. The database stores movies, lead actors, movie genres and screenings.

I created the tables, inserted data and changed the structure of some tables. Then I wrote 9 queries to get information from the database.

2. Database Model

I created a database named “cinema” with four tables. I also created the database model in MySQL Workbench.

Table	What it stores
movie	The title, the year and the duration of each movie
lead_actor	The first and last name of the lead actor of each movie
movie_genres	The genre of each movie
screening	The day, the start time and the movie of each screening

The movie table is the main table. The other three tables are linked to it through foreign keys that reference the movie_id column.

3. Creating the Tables

3.1 The movie table

The movie_id column is the primary key and increases automatically. The movie title is mandatory. The movie_time column stores the duration in minutes.

```
create database cinema;
use cinema;

create table movie (
movie_id int auto_increment primary key,
movie_title varchar(50) not null,
movie_year int,
movie_time int
);
```

I inserted 15 movies, for example Forrest Gump, Titanic, The Matrix and Avatar.

```
insert into movie (movie_title, movie_year, movie_time)
values ('Forrest Gump', 1994, 194), ('Bohemian Rapsody', 2018, 205), ('Titanic',
1997, 190), ...
```

3.2 The lead_actor table

This table stores the lead actor of each movie. The actor_id column is the primary key. It is also a foreign key that references the movie table.

```
create table lead_actor (
actor_id int auto_increment primary key,
first_name varchar(50),
last_name varchar(50),
movie_id int not null,
foreign key (actor_id) references movie(movie_id)
);
```

I inserted 15 records, one for each movie. Some actors appear in more than one movie: Tom Hanks in four movies and Leonardo DiCaprio in three.

3.3 The movie_genres table

This table stores the genre of each movie. The movie_ref_id column is a foreign key that references the movie table.

```
create table movie_gender (
movie_ref_id int,
gender_type varchar(50),
foreign key (movie_ref_id) references movie(movie_id)
);
```

I inserted one genre for each movie. I used five genres: Drama, Horror, Adventure, SciFi and Romantic.

3.4 The screening table

This table stores the screenings of the cinema: the day, the start time, the duration and the title of the movie.

```
create table screening (
screening_day date,
start_time time,
movie_time int,
movie_title varchar(50)
);
```

I inserted 11 screenings, between 25 and 27 July 2022.

4. Changing the Structure

After creating the tables, I made some changes to their structure.

First, I renamed the movie_gender table to movie_genres. I also renamed the gender_type column to genres_type.

```
alter table movie_gender rename to movie_genres;
alter table movie_genres rename column gender_type to genres_type;
```

Then I linked the screening table to the movie table. I added a new column, screening_id. I filled it in with the id of each movie, using UPDATE statements. At the end, I added the foreign key.

```
alter table screening add screening_id int not null;

update screening set screening_id = 2 where movie_title = 'Bohemian Rapsody';
...

alter table screening add foreign key (screening_id) references movie(movie_id);
```

After each change, I verified the tables with SELECT * and DESC statements.

5. Queries

I wrote 9 queries. Each task, its query and a short explanation are shown below.

Task 1: Find all movies that appeared after 2000 and have over 3 hours.

```
select * from movie where movie_year > '2000' and movie_time > '180';
```

I combined two conditions with AND. The query returns two movies: Bohemian Rapsody and Catch Me If You Can.

Task 2: Find the name of the movie with the longest duration.

```
select movie_title from movie
where movie_time = (select max(movie_time) from movie);
```

I used a subquery with MAX. Two movies have the same longest duration, so the query returns both: Bohemian Rapsody and Sleepless In Seattle.

Task 3: Find the oldest movie.

```
select movie_title from movie order by movie_year asc limit 1;
```

I sorted the movies by year and kept only the first row with LIMIT 1. The result is Psycho (1960).

Task 4: Find all movies and their year in which Tom Hanks played.

```
select movie_title, movie_year from movie
left join lead_actor on movie.movie_id = lead_actor.actor_id
where lead_actor.first_name = 'Tom' and lead_actor.last_name = 'Hanks';
```

I joined the movie and lead_actor tables and filtered by the actor's name. The query returns four movies: Forrest Gump, Appolo 13, Sleepless In Seattle and The Terminal.

Task 5: Get the genre of the movies in which Leonardo DiCaprio played.

```
select genres_type
from movie_genres left join lead_actor on movie_genres.movie_ref_id =
lead_actor.actor_id
where lead_actor.first_name = 'Leonardo' and lead_actor.last_name = 'DiCaprio';
```

I joined the movie_genres and lead_actor tables and filtered by the actor's name. The result shows the genres Drama and Adventure.

Task 6: Show all distinct movie genres.

```
select * from movie_genres
group by genres_type;
```

I grouped the rows by genre, so each genre appears only once.

Task 7: Get all movies which have a screening record in the cinema.

```
select movie_title from screening;
```

I selected the movie titles from the screening table.

Task 8: Get all movie names that are screened on 26–27 July.

```
select movie_title from screening
where screening_day = '2022-07-26' or screening_day = '2022-07-27';
```

I combined two dates with OR.

Task 9: Compute how many movies are screened on 25 July.

```
select count(*) from screening
where screening_day = '2022-07-25';
```

I used the COUNT function. The result is 4.

The complete SQL script can be checked in the project repository (cinema database.sql). The database model is available in cinema_model.mwb.

6. Conclusion

In this exercise, I designed a database with four related tables. I created the tables, inserted data and changed their structure with ALTER TABLE and UPDATE statements. Then I wrote 9 queries using filters, sorting, joins, subqueries, grouping and functions.

This exercise helped me practise the main SQL statements. It also helped me understand how tables are linked through primary and foreign keys.